Police Pullover Bias Investigation

Max Takacs & Sam Friedman

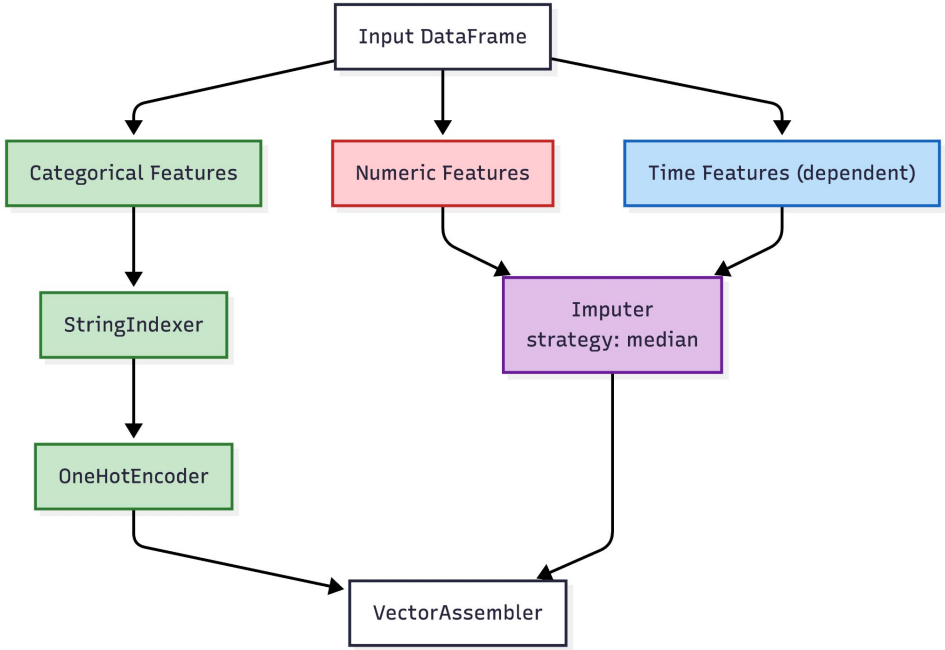
Problem Statement / Objective

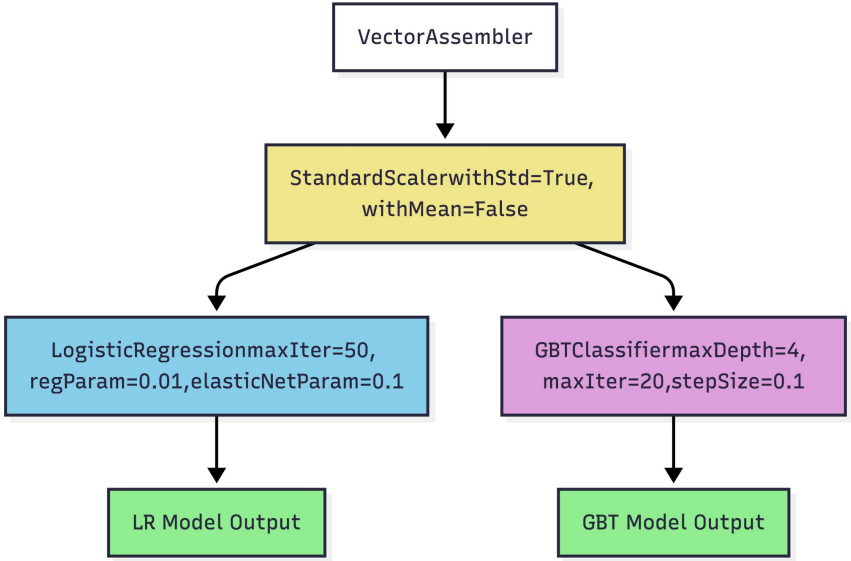
- Using data from the Stanford Open Policing Project and American Community Survey, we built a PySpark pipeline that predicts stop outcomes and gauges fairness to the driver
- This will then be used to compare performance across counties to identify potential bias by officers
- Spark will allow us to scale this to millions of rows effectively and efficiently

THE STANFORD OPEN POLICING PROJECT

AMERICAN COMMUNITY SURVEY
U.S. CENSUS BUREAU

Overview Of Our Solution





Data

NC Pullover Data

20,286,646 Rows

→ Date, Time, Location
→ Subject Data
  → Race, Sex, Age
→ Search Data
  → Conducted, Consent, Contraband
→ Search Outcomes
  → Citation, Warning, Arrest
→ etc.

Left Join

On Year & County

ACS Data

→ Time, Location

→ Median household income

→ Poverty Rate

→ Unemployment Rate

→ Bachelor's Degree Rate

Data

- NC Police data was manually sourced, unzipped, and placed in the GCP bucket for direct usage

```
RAW_URI = f"{BUCKET}/raw/nc_statewide_2020_04_01.csv"
```

- ACS Data was pulled at runtime from the API with a function
  - This was only done on the first run, and subsequent runs made use of the previously sourced data to ensure full utilization of PySpark

```python
def census_get(year: int, dataset_suffix: str, vars_list):
    base = f"https://api.census.gov/data/{year}/acs/acs5{dataset_suffix}"
    params = {
        "get": ",".join(["NAME"] + vars_list),
        "for": "county:*",
        "in": "state:37",  # 37 = North Carolina
    }
    if CENSUS_API_KEY:
        params["key"] = CENSUS_API_KEY
    url = base + "?" + urlencode(params)
    try:
        with urlopen(url) as r:
            data = json.loads(r.read().decode("utf-8"))
    except (HTTPError, URLError) as e:
        print(f"[WARN] ACS fetch failed for {year} {dataset_suffix}: {e}")
        return []
    if not data or len(data) < 2:
        return []
    header = data[0]
    rows = data[1:]
    out = []
    for row in rows:
        obj = dict(zip(header, row))
        obj["_year"] = year
        out.append(obj)
    return out
```

	non_null_count	non_null_pct
raw_row_number	20286645	100.00
date	20286645	100.00
officer_id_hash	20286645	100.00
subject_sex	20286627	100.00
subject_race	20286627	100.00
arrest_made	20286645	100.00
type	20286645	100.00
citation_issued	20286645	100.00
department_name	20286645	100.00
frisk_performed	20286645	100.00
search_conducted	20286645	100.00
search_person	20286645	100.00
warning_issued	20286645	100.00
raw_action_description	20286645	100.00
raw_Ethnicity	20286627	100.00
raw_Race	20286627	100.00
search_vehicle	20286645	100.00
reason_for_stop	20286645	100.00
subject_age	20284199	99.99
location	20231865	99.73
county_name	19856363	97.88
outcome	19681586	97.02
time	10088737	49.73
contraband_found	625014	3.08
contraband_drugs	625014	3.08
contraband_weapons	625014	3.08
reason_for_search	624899	3.08
search_basis	602792	2.97
reason_for_frisk	22211	0.11

Data pre-processing & ETL

Clean Values: Trim strings, standardize NaNs, normalize 0/1 outcomes, clean column names

Time Features: stop_ts, hour, weekend, cyclic (sin/cos) encodings

Join Keys: Normalize county names and map stop year

Aggregate Features: Department stop volume (dept_freq), county-level minority share

Demographic Bucketing: Income quartiles, minority share quartiles

Preprocessing: String indexing, one-hot encoding, imputation, vector assembly, standard scaling

Write Output

→ Partition output by year for fast reads

→ Output file type as parquet

→ Average partition is 3.27 MB with 203,862 rows

ACS Join

```python
result = []
for k, r in by_key.items():
    name = r.get("NAME", "")
    county_norm = name.lower().replace(" county, north carolina", "").strip()
    result.append({
        "acs_year": r["_year"],
        "state": r.get("state"),
        "county_fips3": r.get("county"),
        "county_name_norm": county_norm,
        "median_income": _to_float(r.get("B19013_001E")),
        "poverty_rate": _to_float(r.get("S1701_C03_001E")),
        "unemployment_rate": _to_float(r.get("S2301_C04_001E")),
        "edu_bachelor_pct": _to_float(r.get("DP02_0068PE")),
    })
return result
```

We added:

- median_income
- poverty_rate
- unemployment_rate
- edu_bachelor_pct

```python
rows = fetch_nc_acs(acs_years)
if not rows:
    print("[WARN] No ACS rows; continuing without ACS enrichment.")
    return df

acs_df = spark.createDataFrame(rows)
df = df.withColumn("acs_year",
                   F.when(F.col("year") < 2009, F.lit(2009)).otherwise(F.col("year")))
df = df.join(
    F.broadcast(acs_df),
    on=[df["county_name_norm"] == acs_df["county_name_norm"],
        df["acs_year"] == acs_df["acs_year"]],
    how="left"
).drop(acs_df["county_name_norm"]).drop(acs_df["acs_year"])
```

From these we also created:

- income_quartile
- minority_share_stop - non-white stop share
- minority_share_bucket - quartiles of prev.

Solution

- Two separate binary prediction tasks are trained and evaluated
  - Citations issued
  - Arrests made
- Two datasets are trained on each model
  - With time data
  - Without time date
- Two different predictive algorithms are used
  - Logistic Regression as a baseline
  - GBTClassifier as a more complex model

- Feature selection is done by explicit inclusion to avoid leakage
- Use 80% of data to train and 20% of data to test

Spark Utilization

```python
from pyspark.sql import SparkSession, functions as F, types as T
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler, StandardScaler, Imputer
from pyspark.ml.classification import LogisticRegression, GBTClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.functions import vector_to_array
```

We indexed and OHE all categorical variables

- subject_race
- subject_sex
- reason_for_stop
- type
- county_name

```python
indexers = [StringIndexer(inputCol=c, outputCol=f"{c}_idx", handleInvalid="keep") for c in cat_small]
enc = OneHotEncoder(
    inputCols=[f"{c}_idx" for c in cat_small],
    outputCols=[f"{c}_oh" for c in cat_small]
)
```

```python
imputed_num_cols = []
if num_cols:
    imputed_num_cols = [c + "_imp" for c in num_cols]
    imputer = Imputer(
        inputCols=num_cols,
        outputCols=imputed_num_cols,
        strategy="median"
    )
    stages.append(imputer)
```

For the numeric columns we imputed them to not allow the VectorAssembler to see null values

Spark Utilization cont.

```
assembler = VectorAssembler(
    inputCols=assembler_input_cols,
    outputCol="features_raw",
    handleInvalid="keep"  # just in case anything weird sneaks through
)

scaler = StandardScaler(
    inputCol="features_raw",
    outputCol="features",
    withStd=True,
    withMean=False
)
```

Our Vector Assembler and Scaler

```
lr = LogisticRegression(
    labelCol=label_col,
    featuresCol="features",
    maxIter=50,
    regParam=0.01,
    elasticNetParam=0.1,
    weightCol="class_w"
)

gbt = GBTClassifier(
    labelCol=label_col,
    featuresCol="features",
    maxDepth=4,
    maxIter=20,
    maxBins=32,
    stepSize=0.1,
    subsamplingRate=0.8
)
```

Our two models

- Due to GBTClassifier's complexity we heavily constricted its runs for time efficiency

```
pipe_lr = Pipeline(stages=stages + [assembler, scaler, lr])
pipe_gbt = Pipeline(stages=stages + [assembler, scaler, gbt])
```

Our Pipelines for each model, containing the indexer, encoder, imputer, and everything on this page

Performance Metrics & Report

For Predictive performance: AUROC and AUPR

$$\text{AUROC} = \int_0^1 \text{TPR}(\text{FPR})\, d(\text{FPR}) = \int_0^1 \frac{\text{TP}}{\text{TP}+\text{FN}}\, d\left(\frac{\text{FP}}{\text{FP}+\text{TN}}\right) = \int_0^1 \text{tpr}(\text{fpr})\, d(\text{fpr})$$

$$\text{AUPR} = \int_0^1 \text{Precision}(\text{Recall})\, d(\text{Recall}) = \int_0^1 \frac{\text{TP}}{\text{TP}+\text{FP}}\, d\left(\frac{\text{TP}}{\text{TP}+\text{FN}}\right) = \int_0^1 p(r)\, dr$$

For groupwise fairness (to assess biases): TPR, FPR, and Rate

$$\text{TPR}(\text{group}) = \frac{\text{TP}(\text{group})}{P(\text{group})} = \frac{\text{TP}(\text{group})}{\text{TP}(\text{group}) + \text{FN}(\text{group})}$$

$$\text{FPR}(\text{group}) = \frac{\text{FP}(\text{group})}{N(\text{group})} = \frac{\text{FP}(\text{group})}{\text{FP}(\text{group}) + \text{TN}(\text{group})}$$

$$\text{Rate}(\text{group}) = \frac{\text{Pos}(\text{group})}{P(\text{group}) + N(\text{group})}$$

Tested “fairness” on race, sex, race within income buckets, race within minority share bucket

Model Training

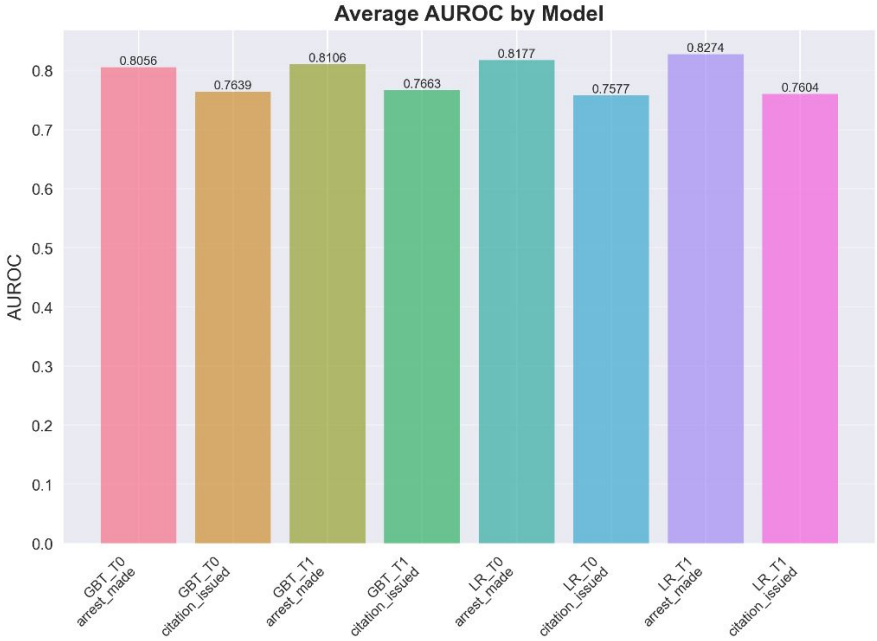
```python
# Class weights
pos = train.filter(F.col(label_col) == 1).count()
neg = train.filter(F.col(label_col) == 0).count()
N   = max(pos + neg, 1)
wpos = N / (2.0 * max(pos, 1))
wneg = N / (2.0 * max(neg, 1))
train = train.withColumn("class_w", F.when(F.col(label_col) == 1, F.lit(wpos)).otherwise(F.lit(wneg)))
test  =  test.withColumn("class_w", F.when(F.col(label_col) == 1, F.lit(wpos)).otherwise(F.lit(wneg)))

# Pipelines: T0 (no time) and T1 (with time)
pipe_lr_T0,  pipe_gbt_T0  = build_pipelines(train, include_time=False, label_col=label_col)
pipe_lr_T1,  pipe_gbt_T1  = build_pipelines(train, include_time=True,  label_col=label_col)
```

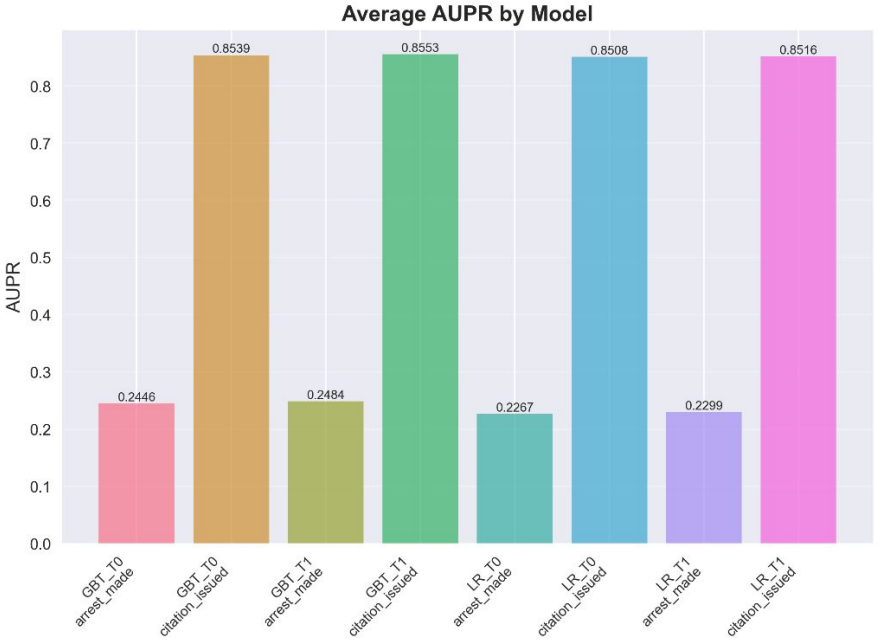
- We added class weights to avoid imbalance in the dataset
- Models were trained with and without time (of day) to compare its impact

```python
eval_roc = BinaryClassificationEvaluator(
    labelCol=label_col,
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"
)
eval_pr  = BinaryClassificationEvaluator(
    labelCol=label_col,
    rawPredictionCol="rawPrediction",
    metricName="areaUnderPR"
)
```

- Evaluation was done using PySpark's BinaryClassificationEvaluator
  - ROC-AUC & PR-AUC were used

Model Performance





Quick Insights ~ Adding time has little effect on performance and LR performs better then GBT

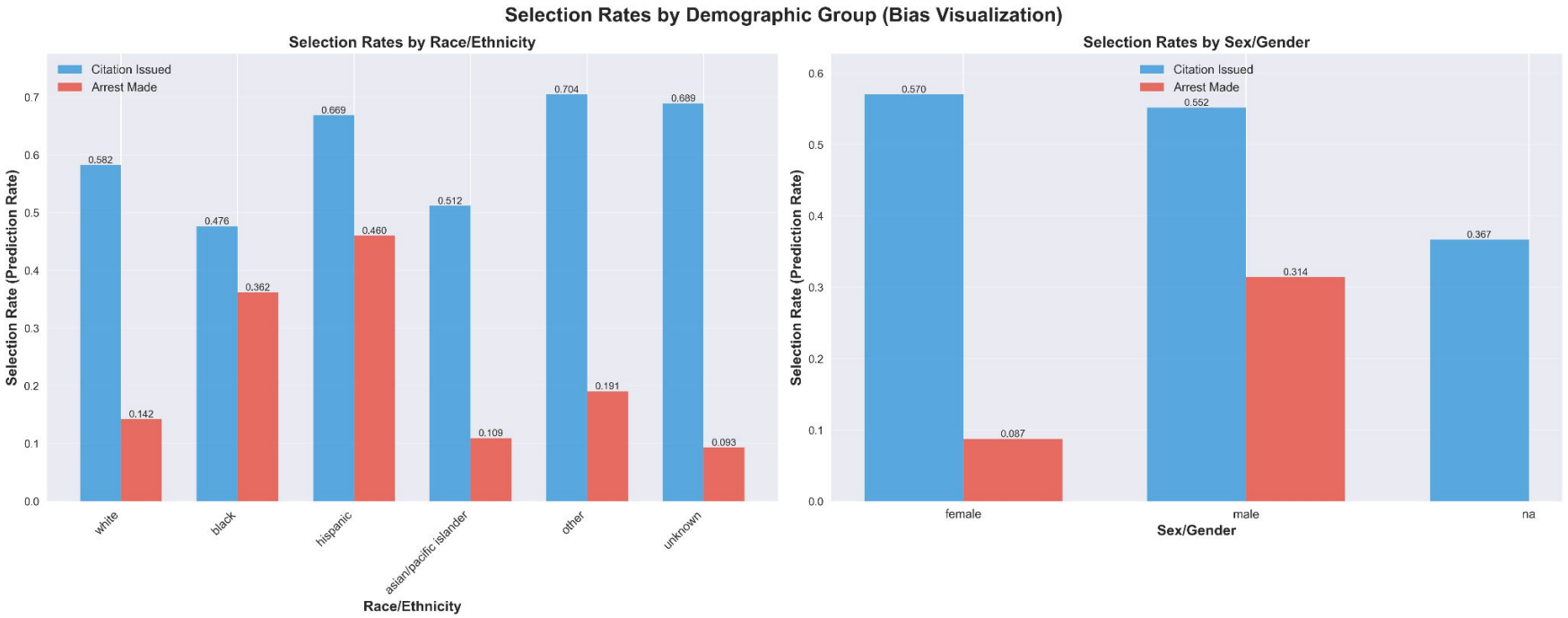
Fairness Evaluation

```python
def fairness_by(df, label_col, pred_col, group_col):
    g = df.groupBy(group_col).agg(
        F.sum(F.when((F.col(label_col) == 1) & (F.col(pred_col) == 1), 1).otherwise(0)).alias("TP"),
        F.sum(F.when(F.col(label_col) == 1, 1).otherwise(0)).alias("P"),
        F.sum(F.when((F.col(label_col) == 0) & (F.col(pred_col) == 1), 1).otherwise(0)).alias("FP"),
        F.sum(F.when(F.col(label_col) == 0, 1).otherwise(0)).alias("N"),
        F.sum(F.when(F.col(pred_col) == 1, 1).otherwise(0)).alias("Pos")
    )
    return (g.withColumn("TPR", F.when(F.col("P") > 0, F.col("TP") / F.col("P")))
             .withColumn("FPR", F.when(F.col("N") > 0, F.col("FP") / F.col("N")))
             .withColumn("Rate", F.when((F.col("P") + F.col("N")) > 0,
                                        F.col("Pos") / (F.col("P") + F.col("N")))))
```

```python
def eval_model(model, name):
    if not model:
        return {}
    pred = (model.transform(test)
            .select(label_col, "prediction", "rawPrediction", "probability", "subject_race", "subject_sex"))
    auroc = eval_roc.evaluate(pred)
    aupr  = eval_pr.evaluate(pred)
    fair_race = fairness_by(pred, label_col, "prediction", "subject_race").toPandas().to_dict(orient="list")
    fair_sex  = fairness_by(pred, label_col, "prediction", "subject_sex").toPandas().to_dict(orient="list")
    return {"model": name, "auroc": auroc, "aupr": aupr,
            "fair_race": fair_race, "fair_sex": fair_sex}
```

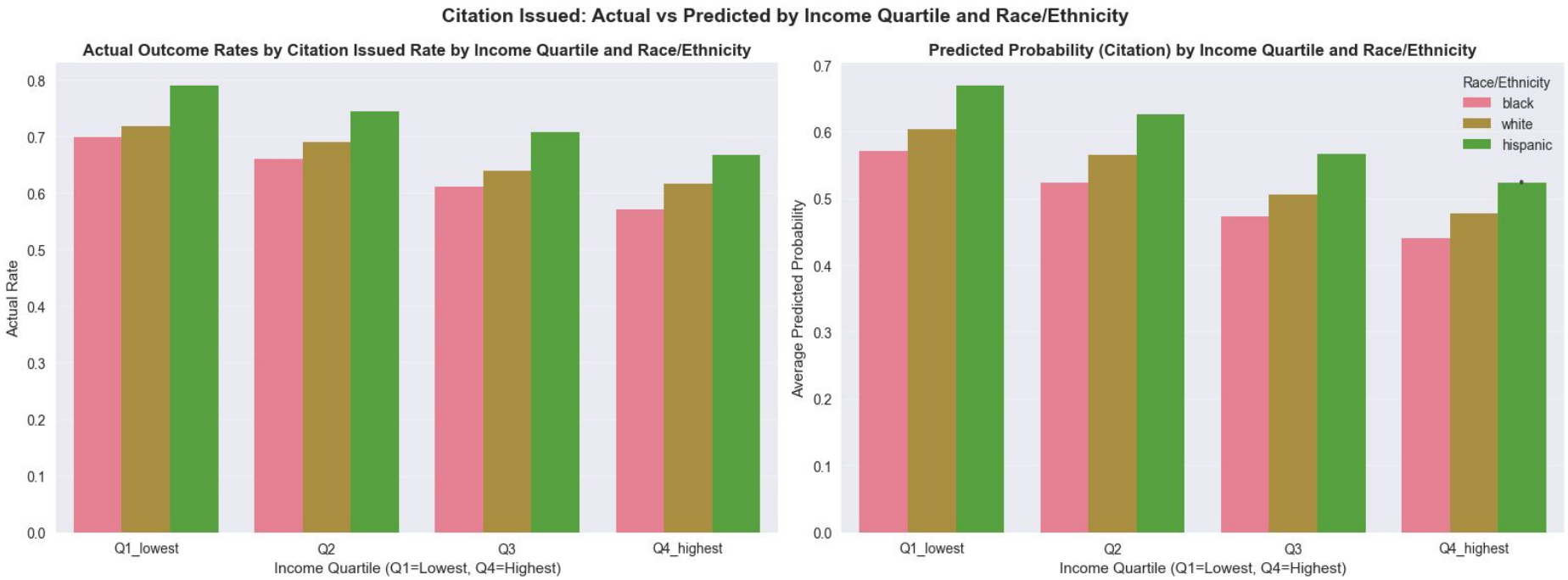
- When evaluating our model, we opted to also evaluate its own fairness toward the demographics in its classifications
- We defined a function to evaluate fairness by a specific group and included this in our main function when evaluating a model's performance

Fairness



Quick Insights ~ This plot shows the models biases when predicting

Income Bias



Quick Insights ~ More money does not mean more problems (the rich are less likely to receive a citation)

Cluster Configuration

Google Cloud

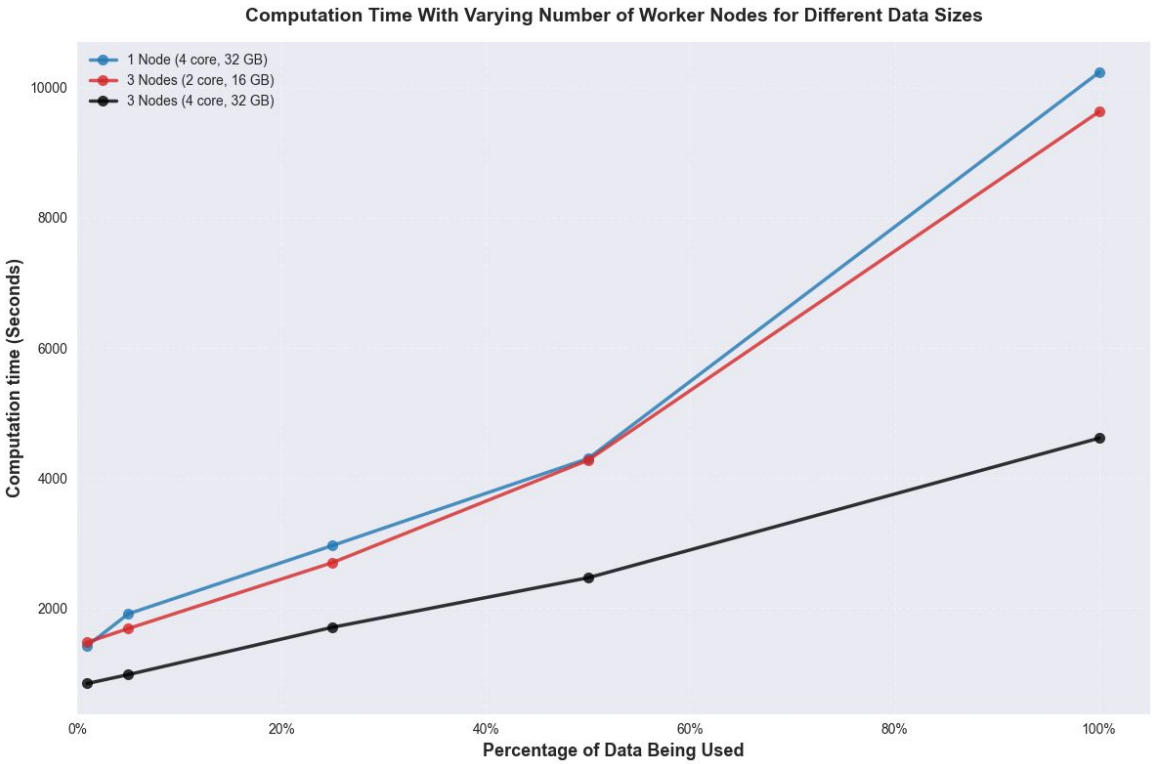
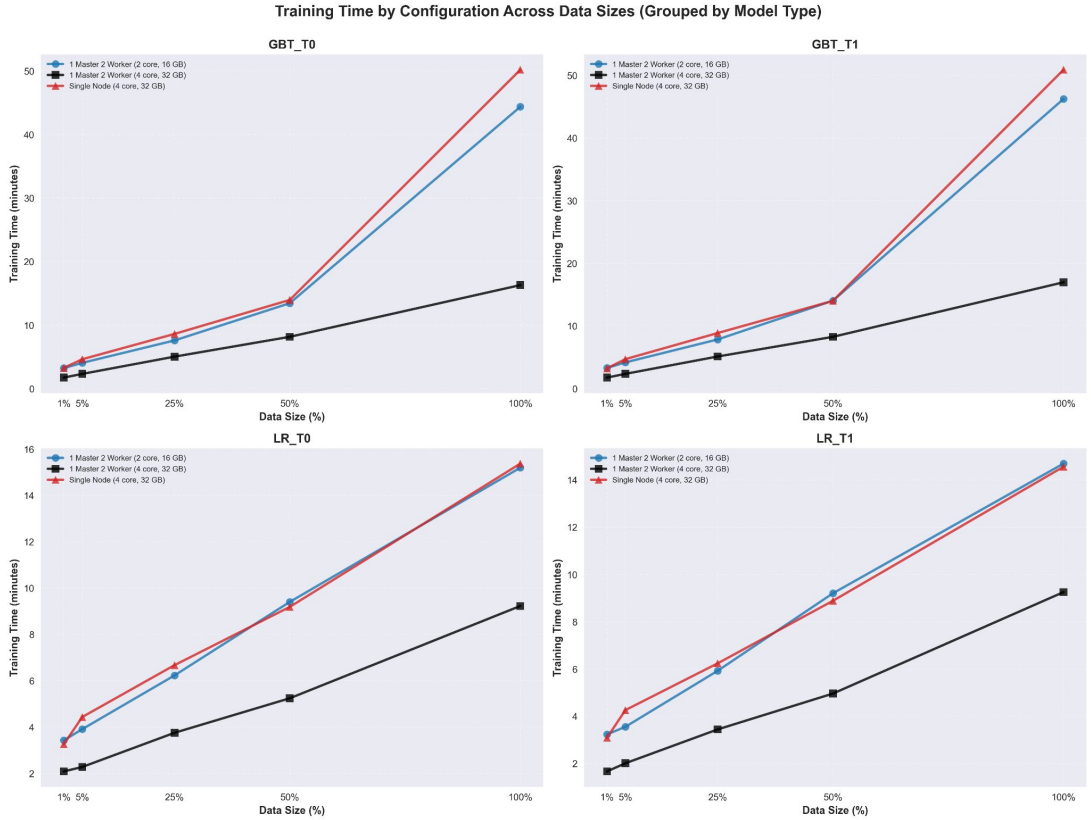
Nodes	**RAM**/Node	**Cores**/Node
Single Node	32 GB	4
1 Master, 2 Workers	32 GB	4
1 Master, 2 Workers	16 GB	2

Cluster Scaling Performance

Data	Config	Cores	RAM	Total (min)	Training (min)	Other (min)
1%	1 Master 2 Worker	2 core	16 GB	24.6	13.2	11.4
1%	Single Node	4 core	32 GB	23.7	12.9	10.9
1%	1 Master 2 Worker	4 core	32 GB	14.0	7.3	6.8
5%	1 Master 2 Worker	2 core	16 GB	28.1	15.7	12.4
5%	Single Node	4 core	32 GB	31.9	18.0	13.8
5%	1 Master 2 Worker	4 core	32 GB	16.3	9.0	7.3
25%	1 Master 2 Worker	2 core	16 GB	45.0	27.6	17.4
25%	Single Node	4 core	32 GB	49.4	30.4	19.0
25%	1 Master 2 Worker	4 core	32 GB	28.4	17.4	11.1
50%	1 Master 2 Worker	2 core	16 GB	71.2	46.1	25.1
50%	Single Node	4 core	32 GB	71.7	46.1	25.6
50%	1 Master 2 Worker	4 core	32 GB	41.1	26.6	14.5
100%	1 Master 2 Worker	2 core	16 GB	160.5	120.5	40.0
100%	Single Node	4 core	32 GB	170.6	131.0	39.6
100%	1 Master 2 Worker	4 core	32 GB	76.9	51.8	25.1

On a model level, LR took the shortest amount of time, and GBT ranged from 2x to 3x the runtime of LR based on cluster configuration

Computation Times With Respect To Node



Computation Times With Respect To Node & Model